Data Structure and Algorithms

Chapter 2: Linked List

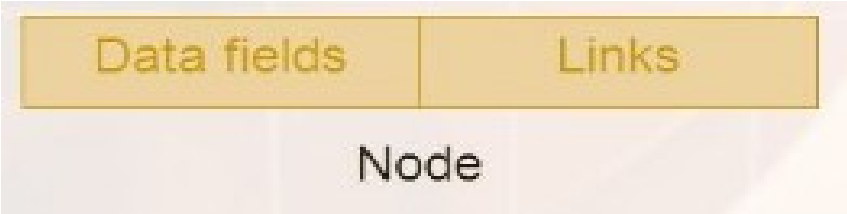
Prepared By: Guta T.

AmiT

March, 2022

Linked List

- A method of organizing stored data in a computers' memory or on storage medium based on the logical order of the data
- All stored data records are assigned a physical address in memory that that the computer uses to locate the information
- A linked list arranges the data by logic rather than by physical address
- Linked list consists of a sequence of nodes, each containing arbitrary data fields and one or two references ("links") pointing to the next and/or previous nodes.



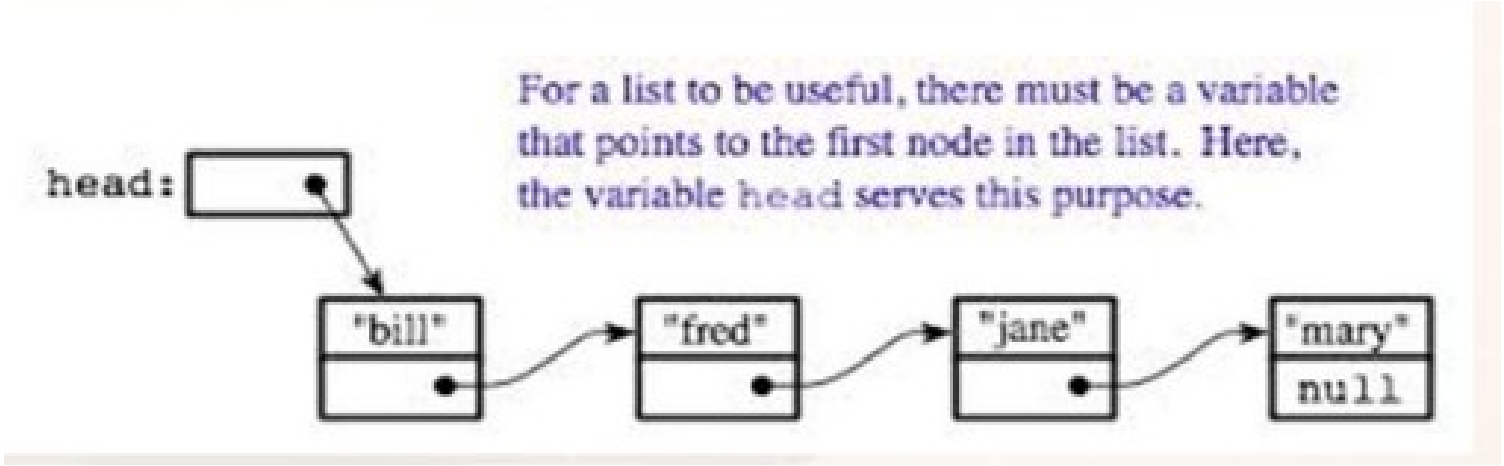
Linked List

- Each node of the linked list has minimum two elements :

  The data member(s) being stored in the list

  A pointer or link to the next element in the list

- The last node in the list contains a NULL pointer to indicate that it is the end or tail of the list
- The last node has a reference to null
- The entry point into a linked list is called the head of the list



Linked List

Address of record	ID	Name	Phone No.	Next Name
0000	1111	Aziz	0199001234	5500
5500	3333	Johan	012444555	6000
6000	4444	Suraya	013909090	8200
8200	5555	Malik	0178000011	eof

Table 1

- In the Table 1, each data record is assigned a memory address
- The first field holds the physical memory address of the record and the last field holds the physical memory address of the next logical record
- The list is said linked because each record is linked to the next based on the last field

Linked List

- What happen if new record is added?
- Assumed with a numerical ID of "2222".
- When this record added to the list, the list change reflect the new linking logic, based on numerical ID

Address of record	ID	Name	Phone No.	Next Name
0000	1111	Aziz	0199001234	9672
9672	2222	Jenab	0157771111	5500
5500	3333	Johan	012444555	6000
6000	4444	Suraya	013909090	8200
8200	5555	Malik	0178000011	eof

Table 2

ARRAYS VS LINKED LISTS

Arrays	Linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access → Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise may result in much memory waste.	Since memory is allocated dynamically(acc. to our need) there is no waste of memory.
Sequential access is faster [Reason: Elements in contiguous memory locations]	Sequential access is slow [Reason: Elements not in contiguous memory locations]

Array Limitations

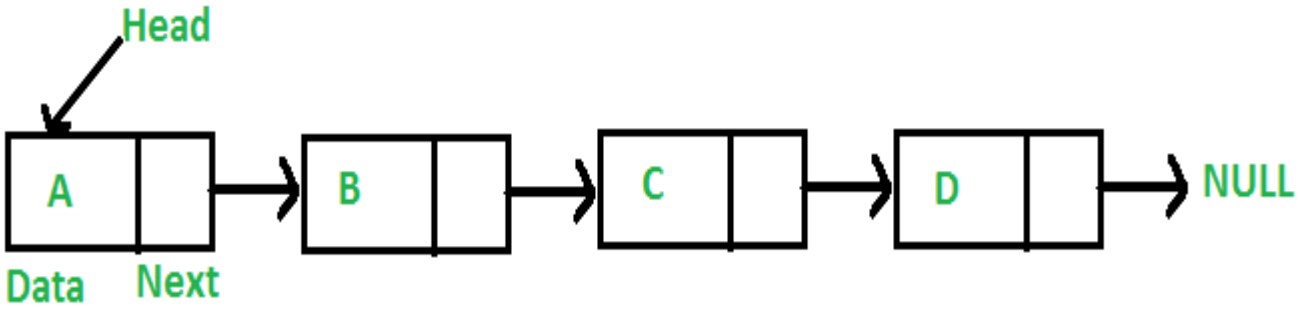
- Arrays are simple to understand and elements of an array are easily accessible, but arrays have some limitations.
- Arrays have a fixed dimension once the size of an array is decided it can not be increased or decreased during education.

Type of Linked List

- Single linked List
- Doubly Linked list
- Circularly Linked list

Singly linked list

- The simplest kind of linked list, which has one link per node
- This link points to the next node in the list, or to a null value or empty list if it is the final node.
- Divided into two parts: the first part holds or points to information about the node, and second part holds the address of next node.
- Travels one way.



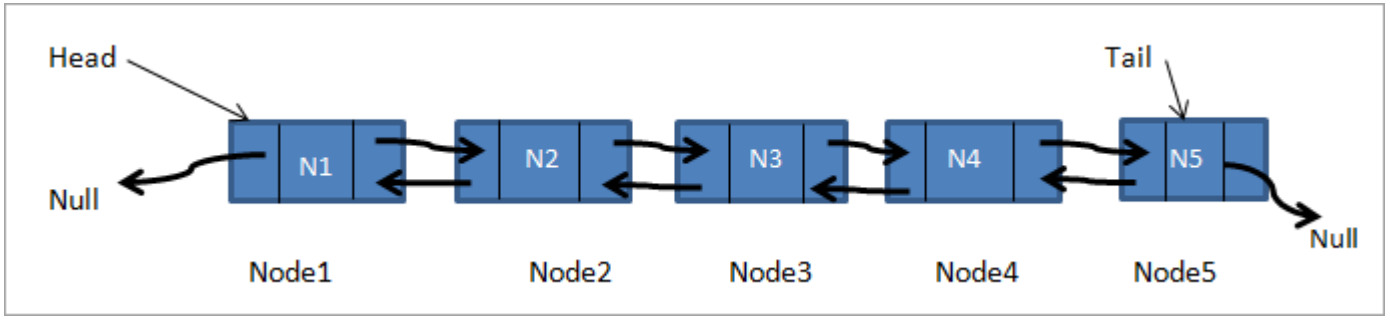
Implementation Singly Linked List

```
Class Node{
public:
int data;
Node* next;
// Default constructor
Node()
{
data = 0;
next = NULL;
}
// Parameterised Constructor
Node(int data)
{
this->data = data;
this->next = NULL;
}
};
```

```
class Linkedlist {
Node* head;
public:
// Default constructor
        Linkedlist() { head = NULL; }
// Function to insert a
// node at the end of the
// linked list.
        void insertNode(int);
// Function to print the
// linked list.
        void printList();
// Function to delete the
// node at given position
            void deleteNode(int);
};
```

Doubly linked list

- Two-way linked list.
- Each node has two links: one points to the previous node, or points to a null value or empty list if it is the first node; and one points to the next, or points to a null value or empty list if it is the final node.



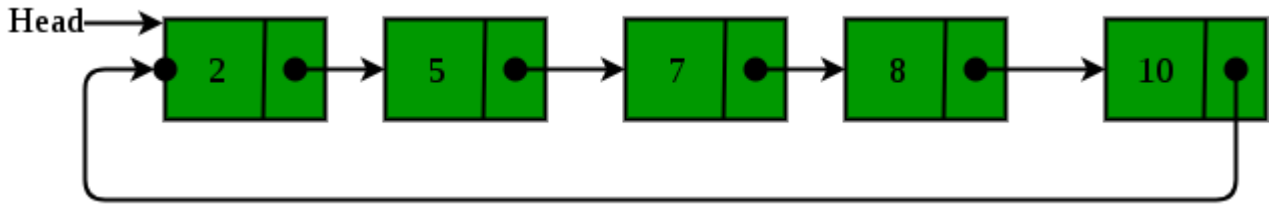
Implementation Doubly Linked List

```
// A linked list node
class Node{
    public:
    int data;
    Node* next;
    Node* prev;
Node(){
data = 0;
next = NULL;
prev=prev;
}
// Parameterised Constructor
Node(int data){
this->data = data;
this->prev=prev;
this->next = NULL;
} };
```

```
class Doubly_Linkedlist {
Node* head;
public:
// Default constructor
      Doubly_Linkedlist() { head =
NULL; }
//inserts node at the front of the list
          void insert_front( int );
/* Given a node as prev_node, insert a
new node after the given node */
          void insert_After(int );
//insert a new node at the end of the list
           void insert_end(int)
 //This function prints contents of linked
list starting from the given node
            void displayList();
};
```

Circularly linked list

- Circular linked list is a linked list where all nodes are connected to form a circle.
- The first and final nodes are linked together.
- There is no NULL at the end.
- A circular linked list can be a singly circular linked list or doubly circular linked list.



Advantages of Circular Linked Lists:

1. Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.
2. Useful for implementation of queue. Unlike this implementation, we don’t need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.
3. Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
4. Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

Basic Operations

- **Insertion** is inserts a new node in the linked list.
  - ✔ **Insertion at front** – Inserts a new node as the first node.
  - ✔ **Insertion at the end** – Inserts a new node at the end as the last node.
  - ✔ **Insertion before a node** –Given a node, inserts a new node before this node.
  - ✔ **Insertion after a node** – Given a node, inserts a new node after this node.
- **Deletion deletes** a node from a given position in the doubly linked list.
  - ✔ **Deletion of the first node** – Deletes the first node in the list
  - ✔ **Deletion of the last node** – Deletes the last node in the list.
  - ✔ **Deletion of a node given the data** – Given the data, the operation matches the data with the node data in the linked list and deletes that node.

Basic Operations..

Traversal

- Traversal is a technique of visiting each node in the linked list.
  - ✔ Forward traversal – Traversal is done using the next pointer which is in the forward direction.
  - ✔ Backward traversal – Traversal is done using the previous pointer which is the backward direction.

Reverse

- This operation reverses the nodes in the doubly linked list so that the first node becomes the last node while the last node becomes the first node.

Search

- Search operation in the doubly linked list is used to search for a particular node in the linked list. For this purpose, we need to traverse the list until a matching data is found.

Summry

- It is easy to insert a node at the front of a list.
- The linked list toolkit also provides a function for inserting a new node elsewhere
- It is easy to remove a node at the front of a list.
- The linked list toolkit also provides a function for removing a node elsewhere--you should read about is function and the other functions of the toolkit.